# PDP-8/IXS Simulator User's Guide[1]

Bill Silver
22 April 2026
Version 1.0

**Summary**

PDP-8/IXS is a Microsoft Windows 10/11 desktop application (C# WinForms) that implements a complete simulation of a Digital Equipment Corporation (DEC) PDP-8/I computer, including:

- Full instruction set
- Interrupts
- Single- and three-cycle data break
- Extended arithmetic element (EAE) KE8/I
- Memory extension control MC8/I and 8 memory fields MM8/I
- RK05 disk drives, 4 units
- DECtape (not yet implemented)
- ASR38/VT100 console terminal (limited escape sequences)
- Front panel with fully functioning lights and switches
- Actual execution speed can match the PDP-8/I at 1.5 μs per cycle.

Every attempt has been made to faithfully simulate the machine as documented by DEC. In addition, PDP-8/IXS emulates custom non-DEC hardware that existed on the MIT Weather Radar Research Project's PDP-8/I in the 1970's, some of which came from the Hurricane Research Project (HRP) of the US Air Force and some added by MIT.

- Index register (MIT)
- Priority interrupt controller (HRP)
- Supervisor call instructions (MIT)
- Real time 8 Hz clock (HRP)
- Interval timer with programmable intervals (MIT)
- Second ASR38/VT100 terminal (MIT)
- Tektronix 611 storage scope (HRP, not yet implemented)
- Radar interfaces for two weather radar sets—digital sweep integrator, azimuth and elevation, antenna servo (HRP & MIT, currently very limited emulation)
- Various displays and switches (HRP)

[1] Copyright (c) 2026 Bill Silver

PDP-8/IXS also has

- Simulated hardware breakpoint, using switch register to break on read or write of a specific address or data word.
- Event recorder for advanced code analysis and debugging
- Standard Windows File menu for creating, opening, and saving a `.pdp8` document that records the complete state of the machine (under development).

**Goals**

- General purpose PDP-8/I simulation
- Running OS/8
- Running E8, the OS/8 Emacs-like editor
- Running the RADAR timesharing system developed at MIT
- Something for a hard-core nerd to do in retirement

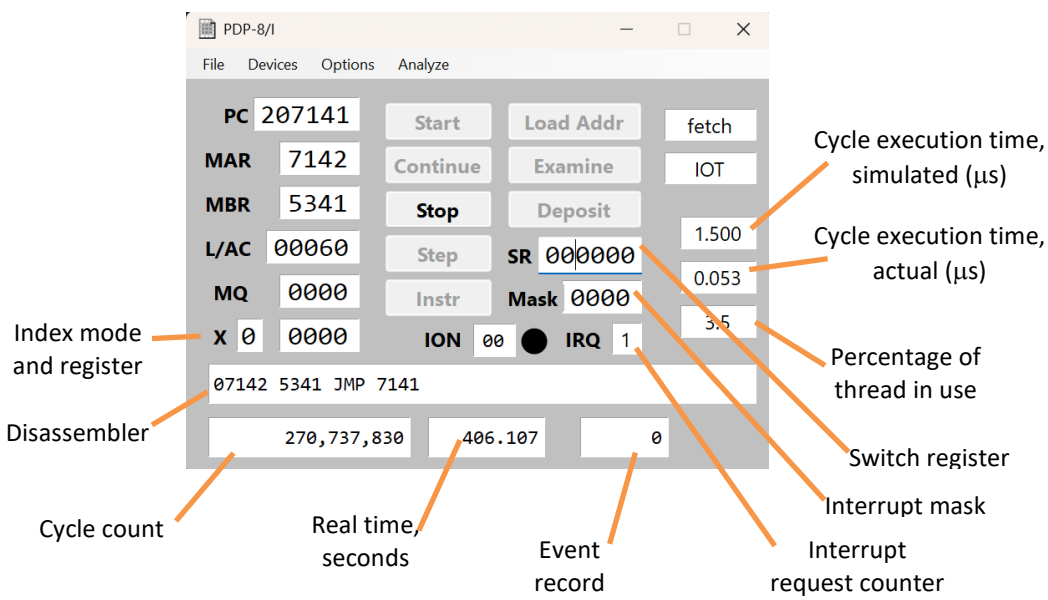
Apologies to the Linux/Mac communities. I'm playing the cards my engineering life has been delt.

**Installation**

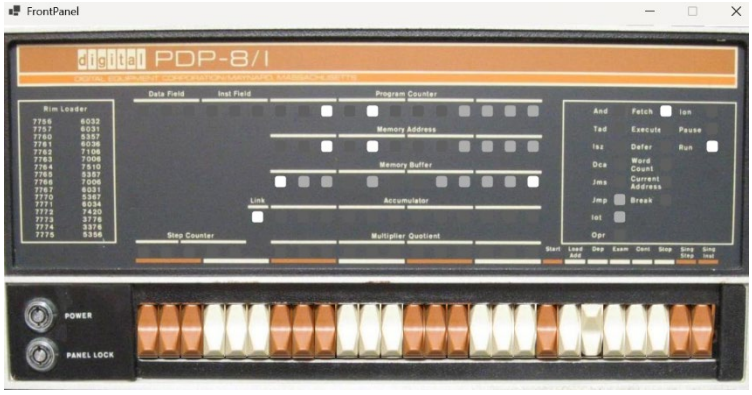
If you have Visual Studio 2022 you can clone the repository and build and run the C# project. You can do as you please with the source code subject to the license in the repository. If you have Windows 10 or 11 you can download the latest release from the repository, a standalone desktop app that has no external dependencies. Just unzip the file and run `PDP-8.exe`.

**Front Panels**

The main window is a modern GUI version of the PDP-8/I front panel, with some additions:



If you select Options→Show Front Panel you'll get this.




The lights are an accurate emulation of incandescent bulbs. All switches are functional and historically accurate. You can click or swipe the switches with a mouse or touchscreen. The switches can be set from either front panel and any changes in one are immediately reflected in the other.

**Single Stepping and Interrupt Suppression**

The following is historically accurate to the best of my knowledge except where noted.

A PDP-8 that is halted is actually quite active. All data break cycles are serviced, so RK05 and DECtape transfers are unaffected by halt. If interrupts are enabled and not suppressed, any interrupt that is not masked off will resume instruction execution to service the interrupt and then continue running. The HLT instruction is useless for debugging when interrupts are enabled.

Interrupts can be suppressed for three reasons:

- When ION is executed, suppress for one instruction to allow interrupt return
- When CIF is executed, suppress until next JMP or JMS
- For breakpoints (not historical) and single instruction stepping.

On a PDP-8/I, when the Single Instr switch was set the CPU halted with interrupts suppressed, and the Continue switch allowed single instruction stepping without interrupt interference. PDP-8/IXS does this, and in addition halts with interrupts suppressed for Sing Step (step one cycle), Stop, and when a breakpoint is hit.

**Front Panel State Displayed**

When the PDP-8 halts the front panels show the state of the CPU at the end of the cycle (which may or may not be the end of a multi-cycle instruction). The PC and disassembler show the next instruction to be executed. The MBR may or may not show the contents of memory at the MAR location, depending on the cycle type.

**PDP-8 Documents (under development)**

A PDP8 document is a file with a `.pdp8` extension that records the complete state of the machine, including all CPU registers and states, all I/O devices, the state of the console switches, and the contents of memory. The contents of the RK05 disks are currently stored in separate files.

The document currently does not store the size and position of PDP-8/IXS windows, but it will at some point. Those sizes and positions are currently stored elsewhere and initialized to whatever they were the last time PDP-8/IXS was run.

The File menu contains the usual Windows commands (New, Open, Save, Save As) for working with documents. If any changes to the machine state are unsaved when exiting PDP-8/IXS, the user will be prompted to save (not yet implemented).

Currently only SaveAs and Open are implemented. SaveAs can be invoked at any time, whether the PDP-8 is running or halted. When the saved document is opened the machine will continue exactly where it was when saved (for now excepting RK05 disks).

A PDP8 document is human-readable XML file and can be examined and edited in any Windows plain-text editor.

### OS/8

File→OS-8 boots OS/8 from RK05 unit 0, surface A by reading block 0 into memory at location 0000, waiting for the read to complete, and then doing JMP 0. If no bootable OS/8 exists on the disc, the results are undefined.

An OS/8 disc image exists in the repository, currently named `v3dNew.rk05`. It was derived from the version of OS/8 recovered by the PiDP-8 project, and contains all of the standard OS/8 commands (e.g. PIP, PAL8, ABSLDR, Fortran) and many historically interesting programs. It also contains E8, an Emacs-like editor that runs under OS/8, which can be used on TTY1 by switching it to VT100 mode (more on this below).

### PAL8 Listing and Binary Files

A listing generated by PAL8 (or any other PDP-8 assembler) has a variety of uses. It can be used to load object code into memory and to show the source code line being executed during single stepping and for breakpoints and recorded events. Currently the listing file must exist on Windows—the OS/8 file can't yet be directly read. I have a Windows app that can inspect and modify an OS/8-formatted RK05 file, and transfer files between Windows and OS/8. I'll make that app available eventually. The PiDP-8 community may have other ways to transfer files.

Listings have `.lst` or `.LS` extensions. The expectation, not enforced, is that `.lst` files came from non-OS/8 assemblers and `.LS` from PAL8 (of course Windows files are not case sensitive). The listing formats are expected to follow PAL8.

A binary file, extension `.bin` or `.BN`, is a listing with just octal address/contents pairs and no side-by-side source code. This is not OS/8 binary format. A binary file can be loaded into core but source line display is unavailable.

| File menu | Description |
|---|---|
| Load Listing | Load listing or binary file. Object code is loaded into core, and if listing file load the file into the Listing window. |
| Split Listing | Split a listing file into three files: `*.bin` (`*.BN`), `*.pal8` (`*.PA`), and `extras.txt`. |
| Write Binary | Extract the binary portion of a listing and write to a file |
| Write Source | Extract the source portion of a listing and write to a file |
| Convert to PAL8 | Convert file to upper case, CR/LF line endings. This command will probably be eliminated in subsequent releases. |

**ASR38/VT100 Terminals**

Devices→TTY1 and Devices→TTY2 open terminal windows. TTY1 is the DEC standard console terminal, IOT addresses 03 (keyboard) and 04 (printer). TTY2 uses corresponding IOT addresses 13 and 14.

Each terminal can choose its simulated print speed by setting the $\log_{10}$(character/sec) control, from teletype speed of 10 cps up to 100,000cps.

The reason ASR38 was chosen instead of ASR33, which was actually DEC standard, is to simulate two-color printing, which the MIT RADAR system used. Two-color is not yet implemented.

In ASR38 mode the terminal remembers all printed output, just as paper would. In VT100 mode the window is a screen of 42 lines of 80 characters. The terminal window remembers the contents of the paper or screen as you switch modes.

The only PDP-8 program that actually cares which mode the terminal is in is E8. Switch to VT100 mode before running it from the OS/8 prompt.

The VT100 screen implements the following ANSI escape sequences:

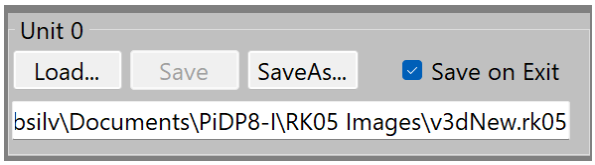
| Sequence | Action |
|---|---|
| ESC [ *row;col* H | set cursor position |
| ESC [ 2 J | clear screen |
| ESC [ K | clear to end of line |
| ESC [ *top;bot* r | set scrolling window |
| ESC D | if cursor is at window bottom scroll window up |
| ESC M | if cursor is at window top scroll window down |
| ESC c | reset terminal (clear screen, reset scroll window) |

The screen will also scroll up if a line feed code ($012_8$) is received when the cursor is at the bottom of the scroll window.

If the ALT key is pressed in combination with any character, the ESC code ($033_8$) is transmitted first.

**RK05 Disks**

Devices→RK05 opens the RK05 window. Controls are shown for all four units. Each looks like this, for example:




The contents of each unit, if any, is loaded from and saved to a Windows file that is compatible with the PiDP-8. The Save button is enabled if any writes to the disk have occurred since it was loaded or saved. Data transfers are by single-cycle breaks.

**Priority Interrupt Controller and Mask**

The HRP priority interrupt system has 16 interrupt channels, 0 – 15, with 15 being the highest priority, and a 12-bit interrupt mask, where a mask bit is set to disable interrupts on the corresponding channel. Here is the PDP-8/IXS configuration.

| Device | IOT Addr | Channel # | Mask Bit |
|---|---|---|---|
| TSVC | 57 | 1 | 9 |
| interval timer | 17 | 2 | 10 |
| clock | 37 | 3 | 11 |
| RK05 | 74 | 4 | none |
| integrator | 33 | 8 | 0 |
| TTY2 printer | 14 | 9 | 1 |
| TTY1 printer | 04 | 10 | 2 |
| PT punch | 02 | 11 | 3 |
| TTY1 keyboard | 03 | 12 | 4 |
| PT reader | 01? | 13 | 5 |
| TTY2 keyboard | 13 | 14 | 6 |
| supervisor call | 57 | 15 | 7 |

Instructions:

| ASM | opcode | operation |
|---|---|---|
| LDMSK | 6561 | AC → mask |
| RDMSK | 6562 | mask → AC |
| RDINT | 6564 | highest priority interrupting channel → AC |

### Supervisor Call

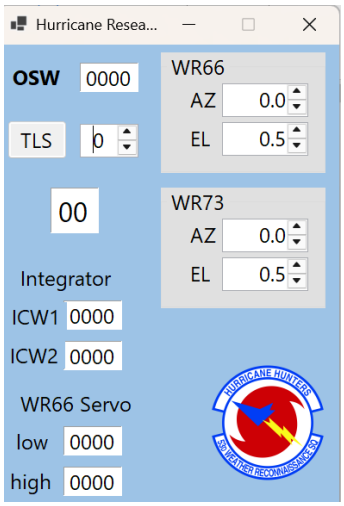
| ASM | opcode | operation |
|---|---|---|
| SVC | 6571 | set SVC interrupt request flag |
| CLSVC | 6573 | clear SVC interrupt request flag |
| XSVC | 6572 | unknown, ignored |
| TSVC | 6574 | toggle TSVC interrupt request flag |

### Clock and Interval Timer

The interval timer instructions can be OR'd in the usual way.

| ASM | opcode | operation |
|---|---|---|
| CLK 2 | 6372 | clear clock interrupt request flag |
| LIT | 6171 | clear interrupt flag, load interval in ms from AC, start timing |
| | 6172 | clear AC |
| LITC | 6173 | clear flag, load from AC and start, then 0 → AC |
| SIT | 6174 | skip if interrupt request flag is set |

### Other MIT and HRP Hardware




Devices→HRP opens this.

Some of these devices are currently of unknown operation and the IOT instructions do little more than display output values to help recover the actual function.

HRP provided a switch to choose one of two radars, which they called side and front (of a C-130). MIT used WR66 (10 cm klystron) and WR73 (5 cm magnetron). WR66 azimuth and elevation readings came from 14-bit synchro-to-digital converters producing binary angles. WR73 AZ and EL come from binary-coded-decimal angles in tenths degree. For the DNS instructions that read WR73 BCD angles (0 is tenths digit, 1 unit, 2 tens, 3 hundreds):

| azimuth | | elevation | |
|---|---|---|---|
| DNS 2 | DNS 7 | DNS 7 | DNS 3 |
| 332222111100 | 00---------- | ----------33 | 222211110000 |

Instructions:

| ASM | opcode | operation |
|---|---|---|
| INA 1 | 6111 | latch S/D (ignored) |
| INA 2 | 6112 | low 12 bits of WR66 AZ → AC |
| INA 3 | 6113 | low 12 bits of WR66 EL → AC |
| INA 7 | 6117 | high 2 bits of WR66 EL (bits 8-9) and AZ (bits 10-11) → AC |
| DNS 2 | 6122 | high bits of WR73 AZ → AC |
| DNS 3 | 6123 | low bits of WR73 AZ → AC |
| DNS 7 | 6127 | low bits of WR73 AZ and high bits of EL → AC |
| FRE 4 | 6314 | release S/D (ignored) |
| SRA 2 | 6322 | load integrator control word 1 (ICW1) |
| SRA 6 | 6326 | load integrator control word 2 (ICW2), including a currently unknown bit that clears the interrupt flag, starts the integrator, and sets the flag 64 ms later. |
| RVI 2 | 6332 | clear integrator interrupt request flag |
| RVI 4 | 6334 | read next range bin from integrator (currently reads Gaussian noise) |
| INB 1 | 6351 | latch/release WR73 angles (ignored) |
| TL2 1 | 6411 | skip if TLS button has been pressed since last TL2 1 |
| TL2 2 | 6412 | 0 → AC |
| TL2 4 | 6414 | AC \|= TLS digit |
| TL2 6 | 6416 | TLS digit → AC |
| OSW 1 | 6451 | AC → OSW, bit 7 is WR66 (0) WR73 (1), others unknown |
| OSW 2 | 6452 | AC → 2-digit 7-segment display, bits 4-7, 8-11 BCD digits |
| RH66 | 6514 | high bits of WR66 servo → AC (all unknown purpose) |
| RL66 | 6515 | low bits of WR66 servo → AC |
| WL66 | 6516 | AC → low bits of WR66 servo |
| WH66 | 6517 | AC → high bits of WR66 servo |

**Execution Speed**

PDP-8/I cycles are tied to the 1.5 μs core memory read-writeback cycle. Core, like DRAM, is destructive read so every cycle reads and writes back, and read/modify/write cycles like ISZ are just normal cycles with data modified before writeback. Instructions take 1.5 μs for every cycle except for IOT, which uses one core cycle to fetch the instruction but takes 4.25 μs[2].

[2] *Small Computer Handbook*, 1970 Edition, page 35

PDP-8/IXS simulates PDP-8/I timing. Cycles take, on average, 1.5 μs. The extra delay of IOT, called the Pause state, is not simulated. Cycles are run in bursts for various efficiency reasons, so only average cycle time is accurate. This is not noticeable to the human observer, and in principle not noticeable to running code.

Options→10x makes the simulated cycle time 0.15 μs, i.e. ten times faster. RK05 transfers also speed up 10x, but I/O operations like printing speed do not.

The main front panel window (see above picture) shows the current target cycle time (1.50 or 0.15), the actual average time the host CPU needs to run a cycle, and the average percent of time that the thread on the host CPU is active. These averages are computed by a low-pass filter with a 1-second time constant.

I/O instruction cycles, such as TTY output, and event recording can run much slower than a normal cycle due to for example lots of GUI screen updating. These increase the workload on the host thread but do not change the cycle frequency as long as the host can keep up. PDP-8/IXS tries to keep the host thread workload under 100%, but there are cases where it can't. Above 100% just means cycle timing is no longer accurate.

**Hardware Breakpoint**

The hardware breakpoint is controlled by the switch register. The high-order three bits (also used for Data Field entry) control the break mode:

| Bit | State | Effect |
|---|---|---|
| 0 | 0 | break on core field and address matching SR bits 3 - 17 |
| | 1 | break on core data matching SR bits 6 - 17 |
| 1 | 0 | don't break on core writes |
| | 1 | break on matching core writes |
| 2 | 0 | don't break on core reads |
| | 1 | break on matching core reads |

If bits 1 and 2 are 0 (off), the breakpoint is disabled. If they are both set, break will occur on a matching read or write (i.e. any access). The one- and three-cycle data breaks are also monitored and can cause a break. Note that addresses are 15 bits, SR 3-17.

The break occurs on a machine cycle boundary, which can be in the middle of an instruction, as if the Single Step switch was set momentarily at just the right time.

Note that on the actual PDP-8/I, every core cycle was a read/modify/write due to the nature of magnetic core. The hardware breakpoint is based on a programmer's view of what is a read or write (or both, e.g. ISZ).

When a break occurs, if event recording is disabled the CPU will halt with interrupts suppressed. Otherwise an event will be recorded and execution continues.

**Event Recording**

When certain events occur during instruction execution, and recording is enabled by the Analyze→Record checkbox, system state information can be recorded. The events are

- A JMS instruction
- A JMP indirect instruction
- An SVC instruction
- An interrupt
- Breakpoint match

The state recorded includes

- Complete CPU state
- Switch register
- State of all I/O flags
- Cycle count and realtime value

When the CPU isn't running you can examine the last 1000 recorded events using the commands in the Analyze menu. The index of the event record being examined is displayed in the main front panel window. A value of n means n events before the last one recorded. Events are in a ring buffer. An event is displayed by setting the main front panel window to show CPU state, SR, cycle count, real time, and the IRQ consequences of the saved I/O flags and interrupt mask. If a listing was loaded the source code line corresponding to the event is highlighted in the Listing window. It shows the instruction being executed, not the next one that the disassembler shows.